



TremPim Test Document

This document outlines the testing methodology, test scenarios, and validation strategies used to evaluate the functionality, reliability, and performance of the TremPim system, a collaborative ride-sharing platform for students.

The platform allows users to register, verify accounts, offer rides, request seats, communicate in ride chat, exchange notifications in real-time, GPS tracking, and submit safety reports. Because the system combines REST APIs, asynchronous tasks, and WebSocket communication, testing must cover both standard user workflows and operational edge cases.

The objective of this document is to ensure all major components, including backend services, frontend flows, authentication, throttling, notifications, and deployment behavior, operate according to design expectations and provide a stable user experience.

Purpose of the Tests

The purpose of this testing process is to verify the system behaves correctly under normal usage and failure scenarios.

Testing validates:

- Functional correctness: every endpoint and workflow performs as intended.
- Usability: users can complete core tasks without confusion.
- Performance: key features respond within acceptable time under moderate load.
- Security: access control and authentication boundaries are enforced.
- Reliability: background jobs and real-time channels remain stable.
- Compatibility: frontend and backend interaction remains consistent.

By documenting this in a structured way, the team gets a repeatable reference for QA cycles, feature regression checks, and release readiness decisions.

Trempim Test Document.....	1
Purpose of the Tests.....	1
Functional Requirements.....	4
Nonfunctional Requirements.....	20
UI Testing.....	21
Testing Run, Build and Deployment.....	22
Exit Criteria.....	23

Functional Requirements

1.1 UC 1 - Register User

Description: A guest wants to create a new account using their personal details and academic email.

Test Case 1: Verify a guest can register with valid details.

- Steps:
 1. Call POST /api/users/register with valid first name, last name, email, password, phone number and password confirmation.
 2. Confirm terms are accepted.
- Expected Result:
 - Registration succeeds.
 - User record is created in the database pending verification state.

Test Case 2: Verify error is raised for duplicate email.

- Steps:
 1. Register a user with email A.
 2. Attempt second registration using email A.
- Expected Result:
 - Registration fails with duplicate email validation error.

Test Case 3: Verify error is raised for invalid email format.

- Steps:
 1. Call registration with malformed email.
- Expected Result:
 - Registration fails with email format validation error.

Test Case 4: Verify error is raised for a non academic email.

- Steps:
 1. Call registration with a non academic email.
- Expected Result:
 - Registration fails with non academic email validation error.

Test Case 5: Verify error is raised for weak password policy violation.

- Steps:
 1. Call registration with a weak password.
- Expected Result:
 - Registration fails and returns password policy hint.

Test Case 6: Verify throttling behavior on repeated registration attempts.

- Steps:
 1. Send registration requests beyond the allowed rate.
- Expected Result:
 - Endpoint returns a throttling response for excess requests - http code 429.

1.2 UC 2 - Verify Email

Description: A newly registered user wants to verify their account via code.

Test Case 1: Verify user can confirm account with correct verification code.

- Steps:
 1. Register new user.
 2. Call POST /api/users/verify-email with correct code (received via email).
- Expected Result:
 - User verification status changes to verified.

Test Case 2: Verify error for incorrect code.

- Steps:
 1. Submit invalid verification code.
- Expected Result:
 - Verification is rejected.

Test Case 3: Verify error for expired code.

- Steps:
 1. Wait until the code expires.
 2. Submit old code.
- Expected Result:
 - Verification fails with expiration message.

Test Case 4: Verify resend verification code endpoint respects cooldown.

- Steps:
 1. Call POST /api/users/verify-email/resend repeatedly.

- Expected Result:
 - Cooldown and throttling rules are enforced.

Test Case 5: Verify token expiry boundary is correctly enforced using time simulation.

- Steps:
 1. Register a new user and capture the verification token.
 2. Advance system time past the token's TTL using time-freeze tooling.
 3. Submit the now-expired token.
- Expected Result:
 - Verification fails with an expiration error, confirming the TTL boundary is precise.

1.3 UC 3 - Login

Description: A verified user wants to log in and access system features.

Test Case 1: Verify successful login with valid credentials.

- Steps:
 1. Call POST /api/users/login with correct email and password.
- Expected Result:
 - Access and refresh tokens are returned.

Test Case 2: Verify login failure with wrong password.

- Steps:
 1. Call login with incorrect password.
- Expected Result:
 - Authentication error returned.

Test Case 3: Verify login failure for unverified account.

- Steps:
 1. Attempt login before email verification.
- Expected Result:
 - Login denied with verification-related error.

Test Case 4: Verify throttling for brute force attempts.

- Steps:
 1. Send repeated failed login attempts above threshold.
- Expected Result:
 - Login endpoint is throttled.

Test Case 5: Verify login is blocked for suspended users.

- Steps:
 1. Suspend a user account via admin action.
 2. Call POST /api/users/login with the suspended user's credentials.
- Expected Result:
 - Login returns HTTP 403 with an error indicating the account is suspended.

Test Case 6: Verify successful login response contains user object.

- Steps:
 1. Call POST /api/users/login with valid credentials.
- Expected Result:
 - Response payload includes both tokens and a user object with expected fields (id, email, name, etc.).

Test Case 7: Verify admin user can log in successfully.

- Steps:
 1. Call POST /api/users/login with valid admin credentials.
- Expected Result:
 - Login succeeds and tokens are returned for the admin account.

1.4 UC 4 - Update User Profile

Description: A logged-in user wants to maintain personal profile data.

Test Case 1: Verify profile update with valid payload.

- Steps:
 1. Call PATCH /api/users/profile with valid profile fields.
- Expected Result:
 - Profile fields are updated and returned correctly.

Test Case 2: Verify partial update behavior.

- Steps:
 1. Update one field only.
- Expected Result:
 - Only targeted field changes while other fields remain intact.

Test Case 3: Verify error for invalid field values.

- Steps:
 1. Send invalid phone number or malformed payload.
- Expected Result:
 - Validation errors are returned.

Test Case 4: Verify unauthorized profile update is blocked.

- Steps:
 1. Call endpoint without access token.
- Expected Result:
 - Request rejected as unauthorized.

1.5 UC 5 - Register as Driver

Description: A user wants to become a verified driver and create rides.

Test Case 1: Verify successful driver application with valid license image.

- Steps:
 1. Call POST /api/users/register-as-driver with required metadata and valid image.
- Expected Result:
 - Application is accepted and marked for verification flow.

Test Case 2: Verify rejection for invalid image type.

- Steps:
 1. Upload unsupported file format.
- Expected Result:
 - Validation error returned.

Test Case 3: Verify behavior when OCR cannot validate details.

- Steps:
 1. Submit blurry or unreadable license image.
- Expected Result:
 - Application remains unverified and requires manual admin action.

Test Case 4: Verify endpoint throttling on repeated submissions.

- Steps:
 1. Send multiple applications above the allowed limit.
- Expected Result:

- Excess requests are throttled.

Test Case 5: Verify uploaded license filename is obfuscated upon storage.

- Steps:
 1. Login with valid credentials as a non driver user.
 2. Submit a valid driver application with a recognizable filename.
- Expected Result:
 - The stored file uses an obfuscated name, not the original filename, preventing enumeration of private assets.

Test Case 6: Verify license file is saved to private (access-controlled) storage.

- Steps:
 1. Login with valid credentials as a non driver user.
 2. Submit a valid driver application with a license image.
- Expected Result:
 - The file is written to the private media directory, not the publicly accessible media root.

Test Case 7: Verify file content is validated via magic byte check to prevent extension spoofing.

- Steps:
 1. Rename a non-image file (e.g., a text file) with a valid image extension (.jpg).
 2. Login with valid credentials as a non driver user.
 3. Submit the spoofed file as the license image.
- Expected Result:
 - Server reads the file's magic bytes and rejects the upload with a validation error.

Test Case 8: Verify re-registration as a driver is rejected if a profile already exists.

- Steps:
 1. Login with valid credentials as a non driver user.
 2. Submit a successful driver application for a user.
 3. Submit a second driver application for the same user.
- Expected Result:
 - Second submission is rejected with an appropriate error indicating an existing application.

Test Case 9: Verify database transaction rolls back atomically when license validation fails.

- Steps:
 1. Login with valid credentials as a non driver user.

- 2. Submit a driver application where the license file fails post-upload validation.
- Expected Result:
 - No partial driver profile record is left in the database; the transaction is fully rolled back.

Test Case 10: Verify endpoint throttling on repeated submissions.

- Steps:
 1. Login with valid credentials as a non driver user.
 2. Submit a driver application with a file whose contents are corrupt or unreadable as an image.
- Expected Result:
 - The server catches the error and returns a handled error response rather than raising an unhandled exception.

Test Case 11: Verify excessive seat count is rejected at schema level.

- Steps:
 1. Login with valid credentials as a non driver user.
 2. Submit a driver registration payload with a maximum passenger count above the permitted limit(Assuming most vehicles are 5 seats including the driver).
- Expected Result:
 - Schema validation rejects the payload before it reaches service logic.

1.6 UC 6 - Admin Driver Verification

Description: An admin reviews and approves or rejects driver applications.

Test Case 1: Verify admin can approve driver.

- Steps:
 1. Admin calls POST /api/users/admin/verify-driver.
- Expected Result:
 - User status changes to verified driver.

Test Case 2: Verify admin can reject driver.

- Steps:
 1. Admin calls POST /api/users/admin/reject-driver with reason.
- Expected Result:
 - Driver request marked rejected.

Test Case 3: Verify non-admin cannot access admin endpoints.

- Steps:
 1. Non-admin user call verification endpoint.
- Expected Result:
 - Access denied.

Test Case 4: Verify appropriate error is raised when verifying or rejecting a non-existent user.

- Steps:
 1. Admin calls the driver verification or rejection endpoint with a user ID that does not exist.
- Expected Result:
 - Service raises a ValueError or equivalent while endpoint returns a 404/400 error.

Test Case 5: Verify a driver can re-submit an application after a rejection.

- Steps:
 1. The automatic system and an admin reject a driver application.
 2. The user submits a new driver application.
- Expected Result:
 - New application is accepted and a fresh driver profile is created in pending state.

1.7 UC 7 - Create Ride

Description: A verified driver creates a ride offer.

Test Case 1: Verify that a driver can create a ride with valid route, time, and capacity.

- Steps:
 1. Call POST /api/rides with valid payload.
- Expected Result:
 - Ride is created in pending status.

Test Case 2: Verify non-driver cannot create a ride.

- Steps:
 1. Non-driver calls create ride endpoint.
- Expected Result:
 - Permission denied.

Test Case 3: Verify invalid capacity is rejected.

- Steps:

1. Submit ride with invalid max passenger value.
- Expected Result:
 - Validation error.

Test Case 4: Verify invalid time format is rejected.

- Steps:
 1. Submit a malformed datetime.
- Expected Result:
 - Validation error.

Test Case 4: Verify unverified drivers cannot access ride creation.

Steps:

1. Log in with a student account that has not been verified as a driver.
2. Navigate to the "My Rides" section.
3. Tap the "Create a Ride" button.

Expected Result:

- Access to the ride creation menu is blocked.
- An error message or prompt appears instructing the user to verify their driver status.

Test Case 5: Verify unverified students (pending approval) cannot create rides.

Steps:

1. Log in with a student account that has not been verified as a student.
2. Navigate to the "My Rides" section.
3. Tap the "Create a Ride" button.

Expected Result:

- Access to the ride creation menu is blocked.
- An error message or prompt appears instructing the user to verify their student status and apply to be a driver in order to create rides.

1.8 UC 8 - Search Rides

Description: A user searches for rides by filters.

Test Case 1: Verify filter search returns matching rides.

- Steps:

1. Seed rides with different routes and times.
 2. Call GET /api/rides/search/by-filters with city/time filters.
- Expected Result:
 - Only matching rides are returned.

Test Case 2: Verify empty result set for unmatched filters.

- Steps:
 1. Search with a route/time that has no matching ride.
- Expected Result:
 - Endpoint returns an empty list and success code.

Test Case 3: Verify result ordering is stable and predictable.

- Steps:
 1. Search with many matches.
- Expected Result:
 - Results follow expected sort logic.

Test Case 4: Verify endpoint compatibility for frontend query shape.

- Steps:
 1. Execute the same query payload format used by mobile client.
- Expected Result:
 - Backend parses all fields and returns expected response.

Test Case 5: Verify unauthenticated access to ride history is rejected.

- Steps:
 1. Call GET /api/rides/my-rides without an Authorization header.
- Expected Result:
 1. Endpoint returns HTTP 401.

Test Case 6: Verify unauthenticated access to upcoming rides is rejected.

- Steps:
 1. Call GET /api/rides/upcoming without an Authorization header.
- Expected Result:
 1. Endpoint returns HTTP 401.

Test Case 7: Verify my-rides and upcoming endpoints return consistent ride sets.

- Steps:
 1. Create rides for a driver.
 2. Call both GET /api/rides/my-rides and GET /api/rides/upcoming for that driver.
- Expected Result:
 1. The set of ride IDs returned is consistent between both endpoints.

Test Case 8: Verify viewing upcoming rides is blocked for unverified users.

- Steps:
 1. Call the upcoming rides endpoint as a user whose account is not yet verified.
- Expected Result:
 1. Request is rejected with an authorization error.

Test Case 9: Verify viewing driver ride history is blocked for non-drivers.

- Steps:
 1. Call the driver history endpoint as a regular passenger user.
- Expected Result:
 1. Request is rejected with a permission error.

1.9 UC 9 - Request to Join Ride

Description: A passenger requests a seat in an existing ride.

Test Case 1: Verify passenger can request join on eligible ride.

- Steps:
 1. Passenger calls POST /api/rides/{ride_id}/request.
- Expected Result:
 - Join request created as pending.

Test Case 2: Verify duplicate join request is blocked.

- Steps:
 1. Send request twice.
- Expected Result:
 - Second request rejected.

Test Case 3: Verify join request blocked when ride is full.

- Steps:
 1. Fill ride capacity.
 2. Send additional requests.

- Expected Result:
 - Request denied due to no available seats.

Test Case 4: Verify join request blocked for cancelled/completed ride.

- Steps:
 1. Set ride to completed or cancelled.
 2. Send join request.
- Expected Result:
 - Request denied due to status.

Test Case 5: Verify driver cannot request to join their own ride.

- Steps:
 1. Login as a driver.
 2. Create a ride.
 3. call POST /api/rides/{ride_id}/request.
- Expected Result:
 1. Request is rejected with HTTP 400 indicating a driver cannot be a passenger in their own ride.

Test Case 6: Verify join request is blocked for unverified users.

- Steps:
 1. Call the join endpoint as a user whose student verification is pending.
- Expected Result:
 1. Request is denied with an authorization error.

Test Case 7: Verify join request is blocked when ride status is cancelled.

- Steps:
 1. Set the ride status to cancelled.
 2. An eligible passenger calls POST /api/rides/{ride_id}/request.
- Expected Result:
 1. Request is rejected because the ride is no longer accepting passengers.

1.10 UC 10 - Approve or Reject Passenger

Description: A driver manages incoming join requests.

Test Case 1: Verify driver approves passenger successfully.

- Steps:

1. Driver calls POST /api/rides/{ride_id}/approve/{user_id}.
- Expected Result:
 - Passenger moves from pending to approved participant list.

Test Case 2: Verify driver rejects passenger request.

- Steps:
 1. Driver calls DELETE /api/rides/{ride_id}/reject/{user_id}.
- Expected Result:
 - Pending request removed or marked rejected.

Test Case 3: Verify another user cannot approve/reject requests.

- Steps:
 1. Non-driver calls approve endpoint.
- Expected Result:
 - Access denied.

Test Case 4: Verify capacity boundary during approval.

- Steps:
 1. Approve users until capacity is reached.
 2. Attempt one more approval.
- Expected Result:
 - Over-capacity approval is rejected.

Test Case 5: Verify system handles ride becoming full during approval (Race Condition).

Steps:

1. Identify a ride with only one available seat.
2. Have two different users submit a "Request to Ride" at approximately the same time.
3. As the driver, attempt to approve both users.

Expected Result:

- The first user is moved to the approved list and the seat count is updated.
- For the second user, the system detects no seats are available.
- The driver is notified that no seats remain, and the second joining request is blocked.

1.11 UC 11 - Leave Ride / Remove Passenger

Description: Participants can leave rides, and drivers can remove participants.

Test Case 1: Verify passengers can leave approved ride.

- Steps:
 1. Passenger calls DELETE /api/rides/{ride_id}/leave.
- Expected Result:
 - Passenger removed from approved list.

Test Case 2: Verify driver can remove passenger.

- Steps:
 1. Driver calls DELETE /api/rides/{ride_id}/remove/{user_id}.
- Expected Result:
 - Passenger removed and relevant notifications emitted.

Test Case 3: Verify users cannot remove themselves via driver-only endpoint.

- Steps:
 1. Passenger calls driver remove endpoint.
- Expected Result:
 - Permission denied.

1.12 UC 12 - Update Ride Details

Description: Driver updates ride route, time, notes, or capacity.

Test Case 1: Verify successful start location update.

- Steps:
 1. Driver calls PATCH /api/rides/{ride_id}/start-location.
- Expected Result:
 - Start location updated.

Test Case 2: Verify successful end location update.

- Steps:
 1. Driver calls PATCH /api/rides/{ride_id}/end-location.
- Expected Result:
 - End location updated.

Test Case 3: Verify successful start time update.

- Steps:
 1. Driver calls PATCH /api/rides/{ride_id}/start-time.
- Expected Result:
 - Start time updated.

Test Case 4: Verify invalid updates are rejected.

- Steps:
 1. Send malformed patch payload.
- Expected Result:
 - Validation error.

Test Case 5: Verify non-owner cannot patch ride.

- Steps:
 1. A user that is not the driver attempts a ride patch.
- Expected Result:
 - Permission denied.

1.13 UC 13 - Ride Status Lifecycle

Description: Driver changes ride state from pending to in progress to completed or cancelled.

Test Case 1: Verify valid status transitions are accepted.

- Steps:
 1. Driver updates status along allowed path.
- Expected Result:
 - Status change succeeds.

Test Case 2: Verify invalid transition is blocked.

- Steps:
 1. Attempt forbidden status jump.
- Expected Result:
 - Status update rejected.

Test Case 3: Verify ride deletion behavior.

- Steps:
 1. Driver deletes ride via DELETE /api/rides/{ride_id}.

- Expected Result:
 - Ride removed or cancelled according to policy.

Test Case 4: Verify repeating a prior ride.

- Steps:
 1. Driver calls POST /api/rides/{ride_id}/repeat.
- Expected Result:
 - New ride created from template data.

Test Case 5: Verify non-owner cannot repeat a ride.

- Steps:
 1. A user who is not the original ride driver calls POST /api/rides/{ride_id}/repeat.
- Expected Result:
 1. Request is rejected with a permission error.

1.14 UC 14 - Ride Chat via HTTP

Description: Ride participants send and manage chat messages.

Test Case 1: Verify that a ride participant can send a chat message.

- Steps:
 1. Call POST /api/rides/{ride_id}/chat with text.
- Expected Result:
 - Message is stored.

Test Case 2: Verify chat history retrieval.

- Steps:
 1. Call GET /api/rides/{ride_id}/chat.
- Expected Result:
 - Ordered message history by time returned.

Test Case 3: Verify message edit flow.

- Steps:
 1. Call PATCH /api/rides/{ride_id}/chat/{message_id}.
- Expected Result:
 - Message text updated.

Test Case 4: Verify message delete flow.

- Steps:
 1. Call DELETE /api/rides/{ride_id}/chat/{message_id}.
- Expected Result:
 - Message removed or marked deleted.

Test Case 5: Verify non-participant cannot read or write chat.

- Steps:
 1. Non-participant calls chat endpoints.
- Expected Result:
 - Access denied.

Test Case 6: Verify empty message content is rejected with a validation error.

- Steps:
 1. Login as a driver.
 2. Create a ride.
 3. Enter the ride chat.
 4. Hit send without inputting an empty or whitespace-only message.
- Expected Result:
 1. Server rejects the message and returns a validation error.

Test Case 7: Verify sending a chat message to a completed ride is rejected.

- Steps:
 1. Set ride status to completed.
 2. Call POST /api/rides/{ride_id}/chat with a valid message.
- Expected Result:
 1. Message is rejected because the ride is no longer active.

Test Case 8: Verify approved ride participant can send and retrieve chat messages via HTTP.

- Steps:
 1. Approve a passenger on a ride.
 2. Approved passenger calls POST /api/rides/{ride_id}/chat with a valid message.
 3. Call GET /api/rides/{ride_id}/chat.
- Expected Result:
 1. Message is stored and appears in the chat history with HTTP 200.

1.15 UC 15 - Ride Chat via WebSocket

Description: Participants exchange live chat messages over a socket.

Test Case 1: Verify socket connection with valid ws ticket.

- Steps:
 1. Request ws ticket from POST /api/users/ws-ticket.
 2. Connect to ride chat socket.
- Expected Result:
 - Connection succeeds and joins ride group.

Test Case 2: Verify socket connection is rejected for invalid ticket.

- Steps:
 1. Connect using invalid or expired ticket.
- Expected Result:
 - Connection denied.

Test Case 3: Verify broadcast reaches all connected participants.

- Steps:
 1. Send message from client A.
 2. Observe clients B and C in same ride.
- Expected Result:
 - All active participants receive event.

Test Case 4: Verify unauthorized user cannot subscribe.

- Steps:
 1. Non-participant connects to ride socket.
- Expected Result:
 - Authorization failure.

Test Case 5: Verify connection is rejected when ride does not exist.

- Steps:
 1. Obtain a valid ws ticket.
 2. Connect to the ride chat socket using a non-existent ride ID.
- Expected Result:
 1. Connection is closed with an appropriate error indicating the ride was not found.

Test Case 6: Verify unauthenticated socket connection is rejected.

- Steps:
 1. Attempt to connect to the ride chat WebSocket without providing a ws ticket.

- Expected Result:
 1. Connection is refused immediately.

Test Case 7: Verify messages are broadcast only to approved participants.

- Steps:
 1. Connect two approved participants and one pending-request user to the same ride socket.
 2. Send a chat message from one of the approved participants.
- Expected Result:
 1. Approved participants receive the broadcast; non-approved users do not.

1.16 UC 16 - Notification Center

Description: A user manages in-app notifications and read state.

Test Case 1: Verify retrieval of all notifications.

- Steps:
 1. Call GET /api/notifications/all.
- Expected Result:
 - Paginated notifications returned.

Test Case 2: Verify retrieval of unread notifications.

- Steps:
 1. Call GET /api/notifications/all_unread.
- Expected Result:
 - Only unread entries returned.

Test Case 3: Verify mark-read operation.

- Steps:
 1. Call POST /api/notifications/mark-read with notification ids.
- Expected Result:
 - Target notifications marked read.

Test Case 4: Verify delete single notification.

- Steps:
 1. Call DELETE /api/notifications/{notification_id}.
- Expected Result:
 - Notification removed.

Test Case 5: Verify clear all notifications operation.

- Steps:
 1. Call DELETE /api/notifications/clear/all.
- Expected Result:
 - User notification list is emptied.

Test Case 6: Verify notification list is paginated with correct default settings.

- Steps:
 1. Create multiple notifications for a user.
 2. Call GET /api/notifications/all without pagination parameters.
- Expected Result:
 1. Response includes a paginated structure with count, next/previous links, and a results array using the default page size.

1.17 UC 17 - Real-Time Notification Socket

Description: User receives push-like events via WebSocket.

Test Case 1: Verify socket connection for authenticated user.

- Steps:
 1. Connect to notification socket channel with valid auth context.
- Expected Result:
 - Connection established for user-specific channel.

Test Case 2: Verify notification event delivery.

- Steps:
 1. Trigger backend action that creates notification.
- Expected Result:
 - Socket event arrives in connected client.

Test Case 3: Verify isolation between users.

- Steps:
 1. Trigger notification for user A.
 2. Monitor user B connection.
- Expected Result:
 - User B does not receive user A events.

Test Case 4: Verify disconnection cleanly removes user from the channel group.

- Steps:
 1. Connect an authenticated user to the notification socket.
 2. Disconnect the user.
- Expected Result:
 1. The server-side channel group removes the user without errors; no orphaned subscriptions remain.

Test Case 5: Verify sending a notification event with an invalid notification ID is handled gracefully.

- Steps:
 1. Connect an authenticated user to the notification socket.
 2. Trigger a notification dispatch referencing a non-existent notification ID.
- Expected Result:
 1. The consumer handles the invalid ID without crashing; no event is forwarded to the client or error is surfaced gracefully.

Test Case 6: Verify unauthenticated socket connection to notification channel is rejected.

- Steps:
 1. Attempt to connect to ws/notifications/{user_id}/ without a valid auth context.
- Expected Result:
 1. Connection is refused.

1.18 UC 18 - User Rating and Review

Description: Users rate each other after shared rides.

Test Case 1: Verify rating submission succeeds.

- Steps:
 1. Call POST /api/users/{user_id}/rate.
- Expected Result:
 - Rating persisted and score updated.

Test Case 2: Verify duplicate rating policy.

- Steps:
 1. Submit same rating scenario repeatedly.
- Expected Result:
 - Duplicate behavior follows business rules.

Test Case 3: Verify rating deletion flow.

- Steps:
 1. Call DELETE /api/users/{user_id}/rate.
- Expected Result:
 - Rating removed and score recalculated.

Test Case 4: Verify unauthorized users cannot rate.

- Steps:
 1. Call rate endpoint without token.
- Expected Result:
 - Unauthorized error.

Test Case 5: Verify rating is rejected if the rater was not a participant in the ride.

- Steps:
 1. User A and User B are not co-participants in any ride.
 2. User A attempts to rate User B referencing a ride they were not part of.
- Expected Result:
 1. Rating submission is rejected with a validation error.

Test Case 6: Verify rating submission is rejected if the ride has not been completed.

- Steps:
 1. Identify a ride in pending or in-progress status.
 2. Attempt to submit a rating between participants.
- Expected Result:
 1. Rating is rejected because ride completion is required before rating is allowed.

Test Case 7: Verify a user cannot submit a rating for themselves.

- Steps:
 1. Authenticated user calls POST /api/users/{own_user_id}/rate.
- Expected Result:
 1. Request is rejected with an error preventing self-rating.

Test Case 8: Verify changing an existing vote correctly updates the aggregate score.

- Steps:
 1. User A rates User B with an upvote.
 2. User A changes the same rating to a downvote.

- Expected Result:
 1. The aggregate score reflects the revised vote; the previous vote is replaced, not accumulated.

Test Case 9: Verify aggregate score is the arithmetic sum of all rating values.

- Steps:
 1. Multiple users rate a driver with a mix of upvotes and downvotes.
- Expected Result:
 1. The driver's aggregate score equals the sum of all individual rating values.

Test Case 10: Verify attempting to delete a non-existent rating is handled gracefully.

- Steps:
 1. Call DELETE /api/users/{user_id}/rate for a rating that does not exist.
- Expected Result:
 1. Endpoint returns an appropriate error rather than raising an unhandled exception.

Test Case 11: Verify user scores are served from cache on repeated reads.

- Steps:
 1. Request user score (triggers cache population).
 2. Request user score again without any rating changes.
- Expected Result:
 1. Second request is fulfilled from cache (cache hit) without a database query.

Test Case 12: Verify cache miss triggers a fresh score calculation and cache population.

- Steps:
 1. Ensure the user score is not in cache (e.g., after cache flush).
 2. Request user score.
- Expected Result:
 1. Score is recalculated from the database and stored in cache for subsequent reads.

1.19 UC 19 - Safety Reporting

Description: User reports misconduct or safety issues.

Test Case 1: Verify report creation with valid content.

- Steps:
 1. Call POST /api/safety/reports with subject user and report text.

- Expected Result:
 - Safety report created successfully.

Test Case 2: Verify validation for missing mandatory fields.

- Steps:
 1. Send report with missing required field.
- Expected Result:
 - Validation error returned.

Test Case 3: Verify rate-limit protection for abuse.

- Steps:
 1. Send reports above limit in throttle window.
- Expected Result:
 - Requests are throttled.

Test Case 4: Verify report can be linked to ride context when provided.

- Steps:
 1. Submit report including ride reference.
- Expected Result:
 - Report stores ride linkage.

Test Case 5: Verify a user cannot submit a safety report against themselves.

- Steps:
 1. Call POST /api/safety/reports with the authenticated user's own ID as the subject.
- Expected Result:
 1. Request is rejected with HTTP 400.

Test Case 6: Verify report submission fails when referencing a non-existent user.

- Steps:
 1. Call POST /api/safety/reports with a subject user ID that does not exist.
- Expected Result:
 1. Request is rejected with HTTP 400.

Test Case 7: Verify suspended users cannot access the safety reporting API.

- Steps:
 1. Suspend a user.
 2. Attempt to call POST /api/safety/reports using the suspended user's token.

- Expected Result:
 1. API returns HTTP 401 or 403, blocking access for the suspended account.

1.20 UC 20 - GPS and Address Utilities

Description: User receives route timing and address suggestions.

Test Case 1: Verify duration between two addresses is returned.

- Steps:
 1. Call GET /api/gps/duration with valid origin and destination.
- Expected Result:
 - Duration value returned with valid format.

Test Case 2: Verify coordinates endpoint resolves valid addresses.

- Steps:
 1. Call GET /api/gps/coordinates with address.
- Expected Result:
 - Latitude and longitude returned.

Test Case 3: Verify city autocomplete returns relevant suggestions.

- Steps:
 1. Call GET /api/gps/autocomplete/city with partial query.
- Expected Result:
 - Relevant city suggestions returned in order.

Test Case 4: Verify street autocomplete is scoped by city when applicable.

- Steps:
 1. Call GET /api/gps/autocomplete/street with city and partial text.
- Expected Result:
 - Streets are filtered correctly.

Test Case 5: Verify graceful handling of unknown addresses.

- Steps:
 1. Query with non-existent address.
- Expected Result:
 - Error or empty response handled cleanly.

Test Case 6: Verify app handles GPS permission denial.

Steps:

1. Open a ride details page.
2. Ensure device GPS permissions are set to "Denied" in the OS settings.
3. Tap the "Send Location" button.

Expected Result:

- The system identifies the Denied state.
- The user is presented with a prompt to enable location services or redirected to settings.

Test Case 7: Verify GPS API endpoints return expected responses through the HTTP layer.

- Steps:
 1. Call each GPS endpoint (duration, coordinates, autocomplete/city, autocomplete/street) with valid parameters via the API.
- Expected Result:
 1. Each endpoint returns HTTP 200 with the expected response shape.

Test Case 8: Verify GPS endpoints return HTTP 422 when required parameters are missing.

- Steps:
 1. Call any GPS endpoint (e.g., /api/gps/duration) without required query parameters.
- Expected Result:
 1. Endpoint returns HTTP 422 Unprocessable Entity with field-level validation errors.

Test Case 9: Verify street autocomplete uses cached city data when available.

- Steps:
 1. Call GET /api/gps/autocomplete/street for a city that has already been resolved and cached.
- Expected Result:
 1. Response is served using the cached city reference, avoiding a redundant external API call.

1.21 UC 21 - User Settings

Description: User manages personal preferences, including notification settings.

Test Case 1: Verify settings retrieval.

- Steps:
 1. Call GET /api/users/settings.
- Expected Result:
 - Settings payload returned with defaults or saved values.

Test Case 2: Verify partial update of notification preferences.

- Steps:
 1. Call PATCH /api/users/settings with one or more preference fields.
- Expected Result:
 - Updated fields persist and are reflected in next read.

Test Case 3: Verify invalid settings key is rejected.

- Steps:
 1. Send unsupported settings field.
- Expected Result:
 - Validation error returned.

Test Case 4: Verify authentication requirement.

- Steps:
 1. Call settings endpoints without token.
- Expected Result:
 - Unauthorized error.

1.22 UC 22 - Admin Console

Description: Admin can suspend/manage users through the admin console.

Test Case 1: Verify admin suspension revokes user login access.

Steps:

1. As an admin, navigate to the admin console to a reported user.
2. Fill in the reasoning and duration, then confirm the suspension.
3. Attempt to log in with the credentials of the newly suspended user.

Expected Result:

- The admin receives a "Suspension Applied" confirmation.
- The user is prevented from logging in.

- An error message is displayed to the user stating they are suspended and provides the reasoning.

Test Case 2: Verify admin can unsuspend a previously suspended user.

- Steps:
 1. Suspend a user via admin action.
 2. Admin calls the unsuspend action for that user.
- Expected Result:
 1. User's active status is restored (is_active = True) and the user can log in again.

Test Case 3: Verify non-admin cannot perform the unsuspend action.

- Steps:
 1. A non-admin authenticated user calls the unsuspend endpoint for another user.
- Expected Result:
 1. Request is rejected with a permission error.

Test Case 4: Verify appropriate error when attempting to unsuspend a non-existent user.

- Steps:
 1. Admin calls the unsuspend action with a user ID that does not exist in the database.
- Expected Result:
 1. Service raises a ValueError; endpoint returns a 404/400 error.

Test Case 5: Verify driver verification and rejection admin actions pass the correct user ID to the service.

- Steps:
 1. Admin selects a driver user in the Django admin and triggers the "verify driver" or "reject driver" action.
- Expected Result:
 1. The correct user ID is passed to the underlying service, and the driver status updates accordingly.

1.23 UC 23 - Email Domain Validation

Description: The system validates that registration emails belong to a recognised academic domain.

Test Case 1: Verify valid university email domain is accepted.

- Steps:

1. Call the email validation service with a known university domain email (e.g., user@bgu.ac.il).
- Expected Result:
 1. Validation passes.

Test Case 2: Verify non-university email (e.g., Gmail) is rejected.

- Steps:
 1. Call the email validation service with a commercial email address.
- Expected Result:
 1. Validation returns false and registration is blocked.

Test Case 3: Verify partial domain match does not bypass university check.

- Steps:
 1. Submit an email such as user@notauniversity.com where the domain partially resembles a valid one.
- Expected Result:
 1. Validation returns false; only exact domain matches are accepted.

Test Case 4: Verify empty or malformed email strings are rejected.

- Steps:
 1. Call the validation service with an empty string.
 2. Call with a string missing the @ sign.
- Expected Result:
 1. Validation returns false for both inputs without raising unhandled exceptions.

1.24 UC 24 - License Image Validation

Description: The system validates image files at the byte level before accepting them as license uploads.

Test Case 1: Verify file without an extension is rejected.

- Steps:
 1. Submit a file with no extension to the image validation service.
- Expected Result:
 1. Validation raises an error indicating a missing or unrecognised extension.

Test Case 2: Verify unsupported file extension is rejected.

- Steps:
 1. Submit a file with an unsupported extension (e.g., .pdf, .bmp).
- Expected Result:
 1. Validation raises an error for the unsupported type.

Test Case 3: Verify file with spoofed extension is rejected via magic byte check.

- Steps:
 1. Rename a non-image file with a supported extension (e.g., .jpg).
 2. Submit it to the validation service.
- Expected Result:
 1. Magic byte inspection detects the mismatch and raises a validation error.

Test Case 4: Verify file exceeding the size limit is rejected.

- Steps:
 1. Submit an image file larger than the configured maximum size.
- Expected Result:
 1. Validation raises an error indicating the file is too large.

Test Case 5: Verify corrupted image data is detected and rejected.

- Steps:
 1. Submit a file that has a valid extension and correct magic bytes but whose body is corrupt.
- Expected Result:
 1. Validation detects the corrupt content and raises an error.

Test Case 6: Verify a valid GIF image passes all validation checks.

- Steps:
 1. Submit a well-formed GIF image to the validation service.
- Expected Result:
 1. Validation passes without errors.

1.25 UC 25 - License OCR Verification

Description: The OCR service extracts and validates driver information from a submitted license image.

Test Case 1: Verify OCR extraction returns expected data fields from a license image.

- Steps:
 1. Call the OCR extraction function with a mocked Vision API response.
- Expected Result:
 1. Result is a dictionary containing the expected keys (name, expiry date, confidence, etc.).

Test Case 2: Verify license with a future expiry date passes validation.

- Steps:
 1. Supply OCR output with an expiry date in the future.
- Expected Result:
 1. Validation passes.

Test Case 3: Verify expired license is detected and fails validation.

- Steps:
 1. Supply OCR output with an expiry date in the past.
- Expected Result:
 1. Validation fails with an expiry-related error.

Test Case 4: Verify driver name is matched using fuzzy comparison.

- Steps:
 1. Supply OCR output where the extracted name closely matches the registered user name (e.g., minor OCR typo).
- Expected Result:
 1. Fuzzy matching accepts the name as valid.

Test Case 5: Verify mismatched driver name fails validation.

- Steps:
 1. Supply OCR output where the extracted name does not match the registered user name.
- Expected Result:
 1. Validation fails indicating a name mismatch.

Test Case 6: Verify confidence score is calculated correctly from OCR output.

- Steps:
 1. Supply OCR output with known field values.
- Expected Result:
 1. The computed confidence score matches the expected value based on extraction quality.

1.26 UC 26 - Background Task Verification (Celery)

Description: Celery workers execute scheduled and triggered background tasks correctly.

Test Case 1: Verify the score reconciliation task corrects mismatched aggregate scores.

- Steps:
 1. Manually set a user's cached or stored aggregate score to an incorrect value.
 2. Run the reconcile_user_scores Celery task.
- Expected Result:
 1. Task recalculates scores from the ratings table and corrects any mismatch.

Test Case 2: Verify driver license OCR task approves the driver when OCR validation passes.

- Steps:
 1. Submit a driver application.
 2. Trigger the verify_driver_license Celery task with a mocked OCR response that passes all checks.
- Expected Result:
 1. Driver profile status is updated to verified.

Test Case 3: Verify driver license OCR task notifies admin when OCR validation fails.

- Steps:
 1. Submit a driver application.
 2. Trigger the verify_driver_license task with a mocked OCR response that fails validation.
- Expected Result:
 1. Driver profile status is not changed to verified; an admin notification email is dispatched.

Test Case 4: Verify driver license OCR task handles unexpected exceptions gracefully.

- Steps:
 1. Trigger the verify_driver_license task where the OCR call raises an exception.
- Expected Result:
 1. Task catches the exception without crashing the worker; appropriate error handling or logging occurs.

Test Case 5: Verify ride status is automatically advanced by the Celery beat scheduler.

- Steps:
 1. Create a ride whose start time has passed.
 2. Allow the scheduled task to run (every 2 minutes).
- Expected Result:
 1. Ride status transitions from pending to in_progress automatically.

Test Case 6: Verify notification email tasks handle empty and mixed recipient lists correctly.

- Steps:
 1. Trigger the send_admin_email task when no admins exist.
 2. Trigger it when admins without email addresses exist.
 3. Trigger it with a mix of valid and invalid admin emails.
- Expected Result:
 1. Task completes without error in all cases; emails are sent only to valid recipients.

Nonfunctional Requirements

2.1 Security

- Verify JWT is required for protected endpoints.
- Verify role-based restrictions (admin and driver-only flows).
- Verify socket authentication via ticket/middleware path.
- Verify unauthorized resource access is denied.
- Verify private media files (driver licenses, profile images) are not accessible without authentication or appropriate access control.

2.2 Rate Limiting and Abuse Prevention

- Verify throttles on register, login, verify-email, resend, ws-ticket, and safety reports.
- Verify user-facing throttle messages are consistent.
- Verify limits reset correctly after the cooldown window.
- Verify the rate limiter correctly extracts client IP from the REMOTE_ADDR field.
- Verify the rate limiter correctly extracts client IP from the X-Forwarded-For header in proxy environments.
- Verify requests within the configured limit are allowed through without throttling.
- Verify requests exceeding the configured limit return HTTP 429.
- Verify separate rate limit counters are maintained per unique IP address so one client's limit does not affect another.
- Verify custom key functions can be applied to the rate limiter for endpoint-specific throttling rules.

2.3 Performance

- Measure search endpoint response times under increasing dataset size.
- Measure chat and notification delivery latency under concurrent clients.
- Validate queue processing time for background tasks at expected workload.
- Verify search latency with Redis caching.
- Verify GPS street autocomplete response time benefits from city-level caching on repeated queries.

2.4 Reliability and Resilience

- Validate behavior during temporary Redis or worker interruption.
- Validate idempotence and safety of retried operations where applicable.
- Validate no data corruption under concurrent ride updates.

2.5 Background Tasks (Celery)

- Validate ride alert matching runs after ride creation.
- Validate periodic tasks and update ride status automatically by time.
- Validate stale location-sharing cleanup task removes expired records.
- Validate score reconciliation task updates aggregate user scores correctly.
- Validate driver license OCR task correctly transitions driver status based on OCR outcome.
- Validate admin notification email task sends only to admins with valid email addresses.

2.6 Data Integrity

- Validate relational consistency between rides, participants, chat, and notifications.
- Validate deleted records are handled according to business rules.
- Validate media and private assets follow expected lifecycle and cleanup logic.

2.7 Compatibility

- Validate mobile frontend calls align with API contract.
- Validate socket events map correctly to frontend event handlers.
- Validate behavior across development and containerized deployment modes.

UI Testing

3.1 Authentication UI

- Verify input validation errors are clear and actionable.
- Verify loading indicators appear during async actions.

- Verify success and failure states return the user to correct flow.

3.2 Driver Onboarding UI

- Verify license upload interactions and error messaging.
- Verify pending/approved/rejected driver status visuals.

3.3 Ride Creation and Search UI

- Verify route and date-time pickers map to backend format.
- Verify search results reflect selected filters.
- Verify empty state and no-network state messages.

3.4 Ride Detail and Participation UI

- Verify join request state transitions in UI.
- Verify participant list updates after approve/reject/remove.
- Verify ride status badges update correctly.

3.5 Chat and Notification UI

- Verify real-time message rendering order and timestamps.
- Verify edited/deleted message states are reflected.
- Verify unread counters and mark-read actions remain synchronized.

3.6 Settings and Profile UI

- Verify settings toggles persist after app restart.
- Verify profile edits are reflected immediately after save.
- Verify profile image upload and fallback avatar behavior.

Testing Run, Build and Deployment

4.1 Backend Test Run

```
Shell  
pytest -v
```

Recommended test environment notes:

- Use in-memory channel layer for tests to avoid Redis dependency.
- Use local memory cache backend for deterministic test behavior.

4.2 Frontend Run

Shell

```
cd trempim-frontend  
npx expo start -c
```

4.3 Optional Tunnel for External Device Testing

Shell

```
ngrok http 8000
```

4.4 Containerized Run

Shell

```
docker compose up -d
```

4.5 Deployment Validation Checklist

- Backend API is reachable.
- Login and token refresh are functional.
- Ride search, create, and join flows are operational.
- WebSocket chat and notifications connect successfully.
- Celery worker consumes queued jobs.
- Media and private file paths are correctly mounted.

Integration and System Tests

This section documents tests that span multiple components, validate full user journeys end-to-end, or stress concurrency and transactional boundaries.

5.1 End-to-End Core Journey

Test Case 1: Verify the full driver-rider journey from ride creation to completion.

Steps:

1. A verified driver creates a ride via POST /api/rides.
2. A verified passenger searches for and finds the ride.
3. Passenger sends a join request via POST /api/rides/{ride_id}/request.
4. Driver approves the passenger via POST /api/rides/{ride_id}/approve/{user_id}.
5. An approved participant sends a chat message via POST /api/rides/{ride_id}/chat.
6. Driver shares their GPS location during the ride.
7. Driver sets ride status to completed.

Expected Result:

- Each step succeeds in sequence with correct HTTP status codes.
- The ride ends in completed status with the participant on the approved list.
- Chat messages and location updates are stored correctly throughout.
- The test confirms integration between ride management, participation, chat, and GPS modules.

5.2 Concurrency and Race Conditions

Test Case 1: Verify only one approval succeeds when two approvals are attempted concurrently for the last available seat.

Steps:

1. Create a ride with exactly one remaining seat.
2. Two threads simultaneously call the approval endpoint for two different pending passengers.

Expected Result:

- Exactly one approval succeeds; the other receives a seat-unavailable error.
- The ride's seat count never goes negative.
- No data corruption or duplicate approvals occur, confirming row-level locking is enforced.

5.3 Schema and Contract Tests

Test Case 1: Verify notification schema resolvers return correct field values.

- Steps:
 1. Instantiate a notification object with known field values.

2. Serialize it using the notification schema.
- Expected Result:
 1. sender_username, sender_full_name, recipient_username, and ride_id resolve to the expected values across all variants (sender present, sender absent, with ride reference, without).

Test Case 2: Verify paginated notification response schema has correct structure.

- Steps:
 1. Build a paginated notification response with known count and data.
- Expected Result:
 1. Schema exposes count, next, previous, and results fields with correct types.

Test Case 3: Verify notification filter schema enforces correct defaults and field types.

- Steps:
 1. Instantiate NotificationFilterSchema with no arguments.
 2. Instantiate with custom values.
- Expected Result:
 1. Defaults match expected values; custom values are stored correctly; field types are validated.

Exit Criteria

Testing is complete when:

- All priority functional test cases pass.
- No blocker or critical defects remain open.
- Regression checks pass for auth, rides, chat, and notifications.
- Nonfunctional baseline checks meet release expectations.